mdserver — Handbuch

Markdown im LAN: HTTP, Seitenlayout, Suche, Control-Port

2026-11-05

Inhaltsverzeichnis

1 Was mdserver ist **5**
1.1 Einführung 5
1.2 Was es tut 5
1.3 Was es nicht tut 5
1.4 Abhängigkeiten 5
1.5 Zusammenfassung 6
2 Starten **7**
2.1 Einführung 7
2.2 Kanonischer Start (LAN hinter Proxy) 7
2.3 Lokal zum Lesen 7
2.4 Module finden 8
2.5 Health 9
2.6 Zusammenfassung 9
3 Optionen **9**
3.1 Einführung 9
3.2 Tabelle 9
3.3 Was im Stil site wirkt 11
3.4 --dotfiles 12
3.5 Zusammenfassung 12
4 Navigation und Theme **12**
4.1 Einführung 12
4.2 Seitenstil 12
4.3 Theme per URL 13
4.4 Kopfleiste in site 13
4.5 Leiste der älteren Stile 13
4.6 Bücher 13
4.7 Startseite eines Buchs 14
4.8 Zusammenfassung 14
5 Seitenlayout und Suche **14**
5.1 Einführung 14
5.2 Aufbau einer Seite 14
5.3 Handbuchseiten 15
5.4 Ordner ohne index.md 15
5.5 Dunkelmodus, Handy, Druck 16
5.6 Fehlende Seite 16
5.7 CSV als Tabelle 16
5.8 Office-Dateien als Seite 17
5.9 Suche 17
5.10 Zusammenfassung 21
6 HTTP-Verhalten **22**
6.1 Einführung 22
6.2 Routing 22
6.3 Dateien 23
6.4 Grenzen 23
6.5 Antwortzeiten 23

6.6 Protokoll 24
6.7 Zusammenfassung 24
7 Control-Port 25
7.1 Einführung 25
7.2 Bedienen 25
7.3 Status 25
7.4 Reload 26
7.5 Was stop tut 26
7.6 systemd (Kennwerte, nicht der Installationsweg) 27
7.7 Zusammenfassung 27
8 TLS 27
8.1 Einführung 27
8.2 Zertifikat bauen 27
8.3 Dateien 27
8.4 Zusammenfassung 28
9 Hinter dem Proxy 28
9.1 Einführung 28
9.2 Bind und Weiterleitung 28
9.3 Pfade und Symlinks 28
9.4 Methoden und Grenzen 29
9.5 Zusammenfassung 29
10 Site aufbauen 29
10.1 Einführung 29
10.2 Mindestbaum aus dem Repo 29
10.3 Start 30
10.4 Empfohlene Site 30
10.5 Demo 31
10.6 Zusammenfassung 31
11 Fallen 31
11.1 Einführung 32
11.2 Nach dem Update nicht mehr im LAN erreichbar 32
11.3 --bind '*' ohne Anmeldung 32
11.4 Hinter dem Proxy 503 bei vielen Nutzern 32
11.5 Suche findet nach dem Start zu wenig 32
11.6 echo stop | nc 32
11.7 SIGTERM statt ExecStop 32
11.8 ?style=sidebar ändert nichts 32
11.9 ?theme=solarized oder ?toc=0 ändert nichts 32
11.10 Die Seiten sehen nach dem Update anders aus 32
11.11 can't find package docir::mdSource 33
11.12 lib/mdserver/ fehlt 33
11.13 Module flach in lib/ 33
11.14 X-Forwarded-For ohne trusted-proxy 33
11.15 Wurzel = Git-Arbeitskopie, --dotfiles 1 33
11.16 server.key unter --root 33
11.17 Clean-URL-Link auf eine Datei ohne .md 33
11.18 Tcl/Tk-Handbuch im Unterordner, Verweise ins Leere 33

11.19 Suche findet eine Datei nicht 34
11.20 Suche findet keine PDF 34
11.21 or findet nichts mehr 34
11.22 Eine CSV- oder Office-Datei lässt sich nicht herunterladen 34
11.23 Datum aus einer XLSX ist eine Zahl 34
11.24 Handbuchseiten heißen alle "NAME" 34
11.25 Älterer Stand: Befehle über die URL 34
11.26 Dateinamen mit Umlauten: 404 34
12 CLI 35
12.1 Einführung 35
12.2 Start und Stop 35
12.3 PDF-Suche 35
12.4 Grenzen 35
12.5 Lokal 35
12.6 Stil und Theme 35
12.7 URLs 36
12.8 Zertifikat 36
12.9 Tcl/Tk-Handbuch 36
13 Glossar 36
14 Fehlermeldungen 37
14.1 Einführung 37
14.2 Start 37
14.3 PDF-Worker 38
14.4 mdctl 39
14.5 HTTP-Status 39
14.6 Zertifikat 40
15 Lizenz 40

1 Was mdserver ist

1.1 Einführung

mdserver **0.4** ist ein HTTP-Dienst in reinem Tcl (kein Tk). Er liefert Markdown-Dateien zur Laufzeit als HTML aus. Eine Coroutine pro Verbindung, non-blocking I/O: ein langsamer Client blockiert die anderen nicht. Alles läuft in einem Thread; eine teure Anfrage hält die anderen auf, bis sie fertig ist (Kapitel *HTTP-Verhalten*, Abschnitt *Antwortzeiten*).

Module: `mdstack/tools/mdserver/lib/mdserver-0.4.tm`, die Suche in `lib/mdserver/search-0.4.tm` und die Office- und CSV-Ansicht in `lib/mdserver/view-0.4.tm`. Startskript: `mdstack/tools/mdserver/mdserver.tcl`.

Dieses Handbuch beschreibt Start und Optionen, Navigation und Themes, Seitenlayout und Suche, das HTTP-Verhalten, den Control-Port, TLS, den Betrieb hinter einem Proxy und den Aufbau einer Site. Die Anhänge sammeln Fallen, Aufrufe, ein Glossar und Fehlermeldungen. Änderungen von Version zu Version stehen in der Paketdoku unter `mdstack/tools/mdserver/`; dieses Buch beschreibt den aktuellen Stand.

1.2 Was es tut

- HTTP immer, HTTPS optional (`tls`)
- Seitenlayout `site` als Vorgabe: Kopfleiste, Seitenleiste, Brotkrumen, "Auf dieser Seite", Ordner als Kacheln, Dunkelmodus
- Volltextsuche über alle `.md` der Wurzel: `/?q=Wörter`, mit tclutils auch über PDFs (ein Treffer je Seite), Office- und CSV-Dateien; mit Suchtipps und Hilfe, wenn nichts gefunden wird
- Office- und CSV-Dateien als Seite statt als Download (mit tclutils)
- Ältere Stile `plain`, `sidebar`, `sticky`, `collapsible` mit Theme und TOC per Option und per URL
- Navigation: Startseite, Gesamt-Index, Bereiche aus den Top-Level-Ordnern
- Range (206), Conditional GET (304), Control-Port
- Statische Dateien (PDF, Bilder, CSS)
- Verzeichnisseite, wenn keine `index.md` liegt
- Buchordner mit `book.tcl`: Kapitel-Navigation, erzeugte Startseite

Dual: Tcl **8.6** und **9.0**.

1.3 Was es nicht tut

Kein Parser eigener Syntax — rendert über mdstack und docir. Kein Reverse-Proxy, keine Anmeldung, keine Firewall. Ohne `--bind` horcht mdserver nur auf `127.0.0.1`; das passt hinter einem Proxy mit Anmeldung. Die LAN-Installation steht in den Büchern *Proxmox LXC nginx* und *Dokumentenserver LAN*.

1.4 Abhängigkeiten

Paket	verlangt	geprüft mit	Pflicht
Tcl	8.6 oder 9	8.6.14, 9.0.4	ja
mdstack::parser	0.2 oder neuer	0.9.2	ja

Paket	verlangt	geprüft mit	Pflicht
mdstack::html	0.1 oder neuer	0.2	ja
docir	0.1	0.1.1	ja (für docir::html)
docir::mdSource	—	0.2	ja (für mdstack::html)
docir::html	—	0.2	ja (für mdstack::html)
docir::diag	—	0.1	ja (für docir::html)
docir::diagram	—	0.1	ja (für docir::html)
mdstack::theme	0.1	0.1	empfohlen
tclutils::tupdftext	0.1	0.1 (tclutils 0.68.0)	nur PDF-Suche
tclutils::tutask	0.2	0.2 (tclutils 0.68.0)	nur PDF-Worker
tclutils::tuodf, tudocx, tuxlsx	0.1	0.2, 0.1, 0.1 (tclutils 0.69.0)	nur Office-Dateien in Suche und Ansicht
tclutils::tucsv	0.2	0.2 (tclutils 0.69.0)	nur CSV als Tabelle
tclutils::tujson	0.1	0.1 (tclutils 0.69.0)	nur Suche als JSON
tclutils::common	0.3	0.3 (tclutils 0.69.0)	nein; PDF-Cache über replaceFile, sonst dasselbe von Hand
tls	—	—	nur HTTPS

Die docir-Module sind keine Kür. `mdstack::html` 0.2 lädt `docir::mdSource` und `docir::html`, und `docir::html` lädt `docir::diag` und `docir::diagram`. Fehlt eines, endet der Start mit `can't find package docir::…` (Kapitel *Fehlermeldungen*).

Ohne `mdstack::theme` läuft der Server, aber `--theme` wirkt nicht. Am Start steht dann eine Warnung.

Alle tclutils-Module sind freiwillig; fehlt eines, fällt nur seine Funktion weg (Kapitel *Seitenlayout und Suche*). `tupdftext` (ab tclutils 0.67.0) liest den Text von PDFs für die Suche, `tutask` startet dafür einen zweiten `tclsh`. `tuodf`, `tudocx` und `tuxlsx` lesen Office-Dateien, `tucsv` zeigt CSV als Tabelle, `tujson` gibt die Suche als JSON aus (alle drei ab tclutils 0.69.0). Ohne tclutils durchsucht mdserver nur Markdown.

HTTPS braucht das Paket `tls`, unter Debian:

```
apt install tcl-tls
```

Wo die Module liegen müssen, zeigt das Kapitel *Starten*.

1.5 Zusammenfassung

- Ein Dienst, ein Prozess, Markdown wird HTML.
- 0.4 ist die Paketfassung aus `package provide mdserver`.
- mdstack allein reicht nicht: fünf docir-Module gehören dazu.

- tclutils ist freiwillig und schaltet PDF-, Office- und CSV-Dateien in Suche und Ansicht ein.
- Suche und Office-/CSV-Ansicht sind eigene Module neben dem Server-Modul.
- Bind Loopback, wenn nginx davor sitzt.

Nächstes Kapitel: Starten

2 Starten

2.1 Einführung

Arbeitsverzeichnis: `mdstack/tools/mdserver/` oder ein Pfad, der `mdserver.tcl` findet. Die Wurzel der Dokumente ist `--root`, nicht das Skriptverzeichnis.

2.2 Kanonischer Start (LAN hinter Proxy)

```
tclsh mdserver.tcl \
    --root /srv/md \
    --port 8080 \
    --bind 127.0.0.1 \
    --trusted-proxy 127.0.0.1 \
    --control 8099 \
    --title Dokumente
```

`--bind 127.0.0.1` ist die Vorgabe; hier steht es, damit die Zeile auch mit einer älteren Fassung dasselbe tut. Ohne `--bind` horcht mdserver nur auf diesem Rechner. Andere Rechner erreichen den Port erst mit `--bind '*'` (alle Schnittstellen) oder einer Adresse. Dann ist er ohne Anmeldung offen, und die Startmeldung sagt es:

```
  HTTP:  0.0.0.0:8080 (alle Schnittstellen)
         http://localhost:8080/
  Achtung: von anderen Rechnern erreichbar (--bind *); ohne Anmeldung davor nur im
    vertrauten Netz
```

Bis Anfang November 2026 hieß ein fehlendes `--bind` alle Schnittstellen. Wer sich darauf verlassen hat, braucht jetzt `--bind '*'`.

Ohne `--style` gilt `site`, das ganze Seitenlayout. Das frühere Aussehen gibt `--style plain`.

2.3 Lokal zum Lesen

```
tclsh mdserver.tcl --root ../../docs --port 8080
```

Dann `http://127.0.0.1:8080/`. Startseite: `index.md` in der Wurzel. Fehlt sie, zeigt `site` die Ordner und Dokumente als Kacheln.

Im Heimnetz, ohne Proxy, für vertraute Rechner:

```
tclsh mdserver.tcl --root ../../docs --port 8080 --bind '*'
```

Die Anführungszeichen braucht die Shell, sonst setzt sie für `*` die Dateinamen des Ordners ein.

2.4 Module finden

Der Launcher `mdserver.tcl` hängt vor dem `package require mdserver 0.4` diese Ordner an den Modulpfad, relativ zum Skript und nur, wenn es sie gibt:

Pfad	Inhalt
lib	mdserver-0.4.tm, mdserver/search-0.4.tm, mdserver/view-0.4.tm und pdftext-worker.tcl, bei einer Installation alle Module
../lib	Alternative für eine eigene Ablage
../../lib	das lib/ von mdstack (mdstack::parser, ::html, ::theme)
../../../docir/lib/tm	ein docir-Repo neben mdstack

Aus dem Repo läuft der Server damit ohne `TCLLIBPATH`, wenn `docir/` neben `mdstack/` liegt:

```
arbeit/
  mdstack/
    lib/mdstack/...
    tools/mdserver/mdserver.tcl
  docir/
    lib/tm/docir-0.1.1.tm
    lib/tm/docir/...
```

Für eine Installation außerhalb des Repos gehört alles in das `lib/` neben dem Skript. Genau diese Dateien, kein ganzes mdstack:

```
mdserver/
  mdserver.tcl
  mdctl.tcl
  mkcert.tcl
  styles/
    site.css
    sidebar.css
    sticky-top.css
    collapsible.css
  lib/
    mdserver-0.4.tm
    mdserver/
      search-0.4.tm
      view-0.4.tm
    pdftext-worker.tcl
    mdstack/
      parser-0.9.2.tm
      html-0.2.tm
      theme-0.1.tm
    docir-0.1.1.tm
    docir/
      mdSource-0.2.tm
      html-0.2.tm
      diag-0.1.tm
      diagram-0.1.tm
```

Die Unterordner `mdserver/`, `mdstack/` und `docir/` sind Teil des Paketnamens: für `docir::html` sucht Tcl die Datei `docir/html-0.2.tm`, für `mdserver::view` die Datei `mdserver/view-0.4.tm`. Flach nebeneinander gelegt findet `package require` sie nicht.

Für die PDF-Suche kommt tclutils dazu. Der Launcher sucht es nicht selbst: entweder liegen `tclutils/common-0.3.tm`, `tclutils/tupdftext-0.1.tm` und `tclutils/tutask-0.2.tm` ebenfalls in `lib/` (`tupdftext` und `tutask` laden `common`), für Office-Dateien, CSV und JSON dazu `tuzip`, `tuodf`, `tudocx`, `tuxlsx`, `tucsv` und `tujson`, oder die Umgebungsvariable `TCLUTILS_TM` zeigt auf das Modulverzeichnis einer tclutils-Ablage (das Verzeichnis, das den Ordner `tclutils/` enthält):

```
TCLUTILS_TM=/opt/tclutils/lib/tm tclsh mdserver.tcl --root /srv/md
```

2.5 Health

Standardpfad `/__mdserver/health`. Für eine Probe hinter nginx oder in einer Unit. Antwort `200 ok`, oder `503 root unreadable`, wenn `--root` kein lesbares Verzeichnis mehr ist.

2.6 Zusammenfassung

- `--root` ist der Dokumentenbaum.
- Hinter Proxy: Loopback, trusted-proxy, Control-Port.
- Ohne `--bind` nur dieser Rechner; `--bind '*'` öffnet bewusst für das Netz, und die Startmeldung warnt.
- Installation: alle Module in `lib/` neben dem Skript, mit den Unterordnern `mdserver/`, `mdstack/` und `docir/`.
- PDF-Suche: tclutils in `lib/` oder über `TCLUTILS_TM`.

Nächstes Kapitel: Optionen

3 Optionen

3.1 Einführung

`tclsh mdserver.tcl [Optionen]`. Die Vorgaben sind die vorsichtigen: `--bind 127.0.0.1` (nur dieser Rechner), `--trusted-proxy` leer (`X-Forwarded-For` wird ignoriert), dazu Grenzen für offene Verbindungen.

3.2 Tabelle

Option	Standard	Bedeutung
--port	8080	HTTP-Port
--bind	127.0.0.1	Horchadresse. *, leer oder 0.0.0.0 = alle Schnittstellen; die Startmeldung warnt dann

Option	Standard	Bedeutung
--maxconn	200	offene Verbindungen insgesamt; darüber sofort 503 mit Retry-After: 1. 0 = ohne Grenze
--maxperaddr	32	offene Verbindungen je Absenderadresse; darüber 503. Adressen aus --trusted-proxy zählen nicht. 0 = ohne Grenze
--dotfiles	0	.git, .env ausliefern
--pdf-search	auto	PDFs in der Suche: auto (wenn tclutils::tupdftext ladbar), 1, 0
--pdf-cache	$HOME/.cache/mdserver-pdf	Cache für den PDF-Text, außerhalb der Wurzel. Ohne HOME unter dem Arbeitsverzeichnis
--pdf-worker	1	PDFs in einem zweiten tclsh lesen; 0: im Serverprozess
--office-search	auto	.odt, .ods, .odp, .docx, .xlsx in der Suche: auto (jedes Format, dessen Modul ladbar ist), 1, 0
--csv-view	auto	.csv als Tabellenseite und in der Suche: auto (wenn tclutils::tucsv 0.2 ladbar), 1, 0
--office-view	auto	Office-Dateien als Seite statt als Download: auto (jedes Format, dessen Modul ladbar ist), 1, 0
--viewcache	32	MB für fertige Office-und CSV-Seiten; eine Seite bleibt, bis sich Änderungszeit oder Größe der Datei ändern. 0 = aus
--trusted-proxy	leer	Gegenstellen, deren X-Forwarded-For gilt. Exakte Adresse, keine Maske
--maxline	8190	Bytes je Zeile. Anfragezeile darüber: 414, Kopfzeile darüber: 431. 0 = ohne Grenze
--maxheader	16384	Bytes aller Köpfe; darüber 431. 0 = ohne Grenze
--healthpath	/__mdserver/health	Health-Pfad

Option	Standard	Bedeutung
--statuspath	leer	Status als Text über HTTP (wie mdctl.tcl status); leer = aus. Nur hinter einer Anmeldung
--root	.	Dokumentwurzel
--theme	hell	hell, dunkel, solarized. In site zählt nur dunkel
--style	site	Seitenstil: site, plain, sidebar, sticky, collapsible
--stylesdir	../styles	CSS neben lib/
--navbg --navfg	#2c3e50 #ffffff	Farben der Navi-Leiste, nur ältere Stile
--navmax	6	Sektionen inline; mehr ins Dropdown. 0 = nie klappen. Nur ältere Stile
--title	mdserver	Site-Titel, in site links in der Kopfleiste
--toc	1	Inhaltsverzeichnis oben, nur ältere Stile
--control	leer	Control-Port, nur localhost
--no-log	—	kein Zugriffsprotokoll
--cert --key	leer	TLS
--tlsport	8443	HTTPS-Port
--help	—	Hilfe

Mehrere Proxys: `--trusted-proxy "127.0.0.1 10.0.0.5"`.

Für `--pdf-search`, `--pdf-worker`, `--office-search`, `--office-view` und `--csv-view` gelten auch `off`, `no` und `false` als aus. `--pdf-search 1` warnt beim Start, wenn `tclutils::tupdftext` fehlt; `auto` schaltet die Suche dann still ab.

3.3 Was im Stil `site` wirkt

Der Stil `site` baut die Seite selbst, mit einem Stylesheet (`styles/site.css`). Darum greifen nicht alle Optionen:

Option	in site
--theme dunkel	Dunkelmodus für alle Seiten
--theme hell, solarized	ohne Wirkung; hell oder dunkel nach Systemeinstellung
--toc	ohne Wirkung; "Auf dieser Seite" steht rechts ab drei Überschriften
--navbg --navfg --navmax	ohne Wirkung; site hat eine eigene Kopfleiste

Option	in site
--title	Name links in der Kopfleiste und im Fenstertitel

Theme-Namen, `--toc` und die Leistenfarben gelten für `plain`, `sidebar`, `sticky` und `collapsible`.

3.4 `--dotfiles`

Aus, weil eine Wurzel, die zugleich Arbeitskopie ist, sonst `/.git/config` herausgibt. Die Suche hält sich daran: Punkt-Ordner bleiben bei `--dotfiles 0` aus dem Suchbestand.

3.5 Zusammenfassung

- Die Vorgaben sind vorsichtig: nur `127.0.0.1`, kein vertrauter Proxy, höchstens 200 Verbindungen, 32 je Absender.
- Vorgabestil ist `site`; `--style plain` gibt das alte Aussehen.
- Stile liegen in `tools/mdserver/styles/` neben `lib/`.
- Control-Port ist optional, für systemd-Stop aber der saubere Weg.

Nächstes Kapitel: Navigation und Theme

4 Navigation und Theme

4.1 Einführung

Jede Seite trägt eine Navigation. Wie sie aussieht, entscheidet der Stil. Stil und Theme gelten serverseitig und zusätzlich per URL, ohne Neustart.

4.2 Seitenstil

Stil	Wirkung
site	ganzes Seitenlayout: Kopfleiste mit Suchfeld, Kapitel-oder Ordnerleiste, Brotkrumen, "Auf dieser Seite", Kacheln (Vorgabe)
plain	TOC als Block oben (bis 0.4 Vorgabe)
sidebar	TOC als feste Leiste links
sticky	TOC klebt am oberen Rand
collapsible	TOC einklappbar

```
http://127.0.0.1:8080/doc.md?style=plain
http://127.0.0.1:8080/doc.md?style=sidebar
```

Der Stil site legt kein Stylesheet über eine fertige Seite. Er nimmt den gerenderten Inhalt und baut die Seite darum. Die Einzelheiten stehen im nächsten Kapitel. Die vier älteren Stile hängen dagegen eine CSS-Datei hinter das Standard-CSS.

CSS in `tools/mdserver/styles/`: `site.css`, `sidebar.css`, `sticky-top.css`,

`collapsible.css`. Fehlt die Datei, kommt die Seite ohne Stil — kein Fehler. Anderer Ort: `--stylesdir`.

4.3 Theme per URL

Im Stil `site` zählt nur der Dunkelmodus:

```
http://127.0.0.1:8080/doc.md?theme=dunkel
```

Ohne `theme=dunkel` folgt die Seite der Systemeinstellung des Browsers. Hell erzwingen lässt sich nicht.

In den älteren Stilen gelten die Namen aus `mdstack::theme` und der Schalter `toc`:

```
http://127.0.0.1:8080/doc.md?style=plain&theme=solarized&toc=0
```

Namen: `hell`, `dunkel`, `solarized`. `toc=0` nimmt das Inhaltsverzeichnis oben weg.

4.4 Kopfleiste in `site`

- Links der Site-Titel (`--title`), er führt auf `/`.
- **Start** und **Alle Dokumente** (`/?nav=index`).
- Die Top-Level-Ordner als Bereiche, wenn darunter irgendeine `.md` oder `.pdf` liegt. Beschriftung aus der H1 der `index.md`, sonst Ordnername.
- Rechts das Suchfeld (Kapitel *Seitenlayout und Suche*).

4.5 Leiste der älteren Stile

- **Start** führt auf `/` (`index.md` in der Wurzel).
- **Alle Dokumente**: `/?nav=index` — Baum aller `.md`, Titel aus dem ersten H1, dazu jede `.pdf` mit Dateinamen und "(PDF)".
- Top-Level-Ordner als Sektionen, wenn darunter irgendeine `.md` oder `.pdf` liegt. Ab `navmax` (6) klappen sie in ein Dropdown. `--navmax 0` klappt nie.

Farben: `--navbg`, `--navfg`. Abschalten im Konstruktor: `nav 0`. Diese Optionen wirken nur hier, nicht in `site`.

4.6 Bücher

Ein Ordner mit `book.tcl` oder einem `<!-- bookkit:toc:begin -->` gilt als Buch. Reihenfolge aus `book.tcl`, sonst TOC-Block, sonst `NNN--`Präfix.

	site	**ältere Stile**
Kapitelliste	Seitenleiste links, aktuelles Kapitel markiert	nur sidebar: Leiste links
Übersicht	erster Eintrag der Seitenleiste	Mitte der Kapitelleiste
voriges / nächstes	zwei Karten "Zurück" und "Weiter" am Fuß	Kapitelleiste (zurück, Übersicht, weiter) am Fuß
PDF/HTML	Knöpfe über der Kapitelliste	Links auf der Startseite

4.7 Startseite eines Buchs

Fehlt im Buchordner die `index.md`, erzeugt mdserver die Buch-Startseite aus `book.tcl`. Das gilt in jedem Stil.

1. Titel (`title`), darunter der Untertitel (`subtitle`) kursiv.
2. Der erste Absatz der `README.md` — im Buchordner, sonst im Ordner darüber.
3. **Download:** `PDF` und `HTML`, wenn `<name>.pdf` und `<name>.html` neben dem Buchordner liegen. So legt bookkit sie ab.
4. **Kapitel:** nummerierte Liste mit den H1-Titeln.

Für einen Buchordner `Beispiel-de/` heißen die Downloads `Beispiel-de.pdf` und `Beispiel-de.html` im Ordner darüber.

Eine eigene `index.md` hat Vorrang. `book-webindex.tcl` (bookkit) schreibt eine, wenn gewünscht — nötig ist es nicht mehr.

4.8 Zusammenfassung

- Vorgabe ist `site`; die älteren Stile per `--style` oder`?style=`.
- In `site` zählt `theme=dunkel`, sonst die Systemeinstellung.
- `--navbg`, `--navfg`, `--navmax`, `toc` gelten nur für die älteren Stile.
- Ein Buch ohne `index.md` bekommt eine erzeugte Startseite.
- CSS fehlt = ungestylt, nicht 500.

Nächstes Kapitel: Seitenlayout und Suche

5 Seitenlayout und Suche

5.1 Einführung

`site` ist der Vorgabestil. Er baut jede Seite gleich auf, mit einem einzigen Stylesheet (`styles/site.css`), ohne JavaScript. Dazu kommt eine Volltextsuche über alle Markdown-Dateien der Wurzel, mit tclutils auch über PDFs, Office- und CSV-Dateien. Office- und CSV-Dateien zeigt mdserver mit tclutils auch als Seite.

Warum ein eigener Stil: die älteren machten `<body>` zum Raster. Eine lange Seitenleiste schob den Text dann unter eine Lücke ihrer Höhe.

5.2 Aufbau einer Seite

```
+-------------------------------------------------------------+
| [=] Titel  Start  Alle Dokumente  Bereich …        [Suchen]  |
+--------------+--------------------------------+-------------+
| Seitenleiste | Start > Ordner > Seite         | Auf dieser  |
|  Buch:       |                                | Seite       |
|  Übersicht   | # Überschrift                  |  - h2       |
|  Kapitel 1   | Text …                         |    - h3     |
|  Kapitel 2   |                                |  - h2       |
|              | [← Zurück]          [Weiter →] |             |
+--------------+--------------------------------+-------------+
```

Teil	Inhalt
Kopfleiste	Site-Titel, Start, Alle Dokumente, Bereiche, Suchfeld
Seitenleiste	im Buch: Titel, PDF/HTML, Übersicht und Kapitel; sonst Unterordner und Dokumente des Ordners
Brotkrumen	Start › Ordnertitel › Seitentitel
Mitte	der gerenderte Inhalt
Zurück/Weiter	zwei Karten, nur in Buchkapiteln
Auf dieser Seite	h2 und h3 der Seite, ab drei Einträgen

Die Brotkrumen nehmen als Beschriftung den Ordnertitel: Buchtitel aus `book.tcl`, sonst H1 der `index.md`, sonst H1 der `README.md`, sonst den Ordnernamen. Haben ein bookkit-Ordner `book-xyz/` und sein Buch `Xyz-de/` denselben Titel, steht er nur einmal da.

Die Spalte "Auf dieser Seite" fehlt bei weniger als drei Überschriften h2/h3. Die Seitenleiste fehlt, wenn der Ordner nichts zu zeigen hat.

5.3 Handbuchseiten

Eine in Markdown umgewandelte Handbuchseite beginnt mit `# NAME` (deutsch auch `# BEZEICHNUNG`) und hat weitere H1 wie `# SYNOPSIS` und `# DESCRIPTION`. mdserver erkennt solche Seiten an der ersten H1 und behandelt sie im Stil `site` so:

Stelle	Wert
Titel, Brotkrumen, Seitenleiste, Kachel, Suchtreffer	CommandName aus dem YAML-Kopf (TIP-700-Form), sonst der Teil der NAME-Zeile vor dem Gedankenstrich (aus grab -Eingaben umlenken wird grab)
"Auf dieser Seite"	h1 und h2 statt h2 und h3, sobald die Seite mehr als eine H1 hat

Ohne YAML-Kopf und ohne NAME-Zeile bleibt der Titel `NAME`. Die Liste *Alle Dokumente* (`/?nav=index`) nimmt weiter die erste H1; dort heißen solche Seiten `NAME`.

5.4 Ordner ohne index.md

Liegt in einem Ordner keine `index.md` und keine `book.tcl`, zeigt `site` den Inhalt als Kacheln. Eine Kachel ist eine Karte mit Titel und kurzem Text. Oben der Ordnertitel, darunter der erste Absatz der `README.md`, wenn es eine gibt. Dann bis zu vier Gruppen:

Gruppe	Was hineinkommt	Kachel zeigt
Bücher	Ordner mit book.tcl, oder ein Ordner, der Bücher direkt enthält	Abzeichen "Buch", Titel, Untertitel, Sprachen
Bereiche	andere Ordner, wenn darunter eine .md oder .pdf liegt	Titel, erster Absatz von index.md oder README.md
Dokumente	.md-Dateien außer index.md, dazu .pdf	H1 und erster Absatz; bei einer PDF Dateiname, "PDF" und Größe
Dateien	alles andere außer .md, .pdf, .tcl, .tsv, .css; eine .csv oder Office-Datei öffnet als Seite	Name, Art, Größe ("Word, 36 KB")

Ein Ordner nur mit PDFs, Office- oder CSV-Dateien zählt als Ordner mit Dokumenten. Er erscheint in der Kopfleiste, in der Seitenleiste und bei den Kacheln. Die Seitenleiste eines Ordners nennt seine PDFs nach den Markdown-Dokumenten, mit dem Zusatz "PDF".

Ein bookkit-Ordner `book-xyz/` mit genau einem Buch darin führt direkt in das Buch. Mit mehreren Ausgaben (`Xyz-de/`, `Xyz-en/`) führt die Kachel in den Ordner, die Sprachen stehen auf der Kachel.

Punkt-Ordner und Punkt-Dateien erscheinen nie. Ein leerer Ordner zeigt "Keine Dokumente gefunden."

Die älteren Stile zeigen stattdessen eine schlichte Liste, gerendert über mdstack.

5.5 Dunkelmodus, Handy, Druck

Dunkel. Der Dunkelmodus folgt der Systemeinstellung (`prefers-color-scheme`). `?theme=dunkel` oder `--theme dunkel` erzwingen ihn.

Schmale Bildschirme. Unter 1180 px Breite fällt die Spalte "Auf dieser Seite" weg. Unter 820 px verschwinden Seitenleiste und die Links der Kopfleiste; links in der Kopfleiste erscheint der Menüknopf (drei Striche, im Bild `[=]`). Er öffnet die Seitenleiste samt Kopfleisten-Links. Das ist ein Kontrollkästchen mit CSS, kein JavaScript.

Druck. Kopfleiste, beide Seitenspalten, Brotkrumen und die Zurück/Weiter-Karten fallen weg. Gedruckt wird der Text.

5.6 Fehlende Seite

Eine unbekannte URL gibt **404** mit einer Seite im selben Layout: "Nicht gefunden", dazu Links auf die Startseite und auf *Alle Dokumente*. Die älteren Stile senden eine schlichte 404-Seite.

5.7 CSV als Tabelle

Ist `tclutils::tucsv` 0.2 ladbar (`--csv-view auto`, Vorgabe), zeigt `site` eine `.csv` als

Tabelle statt sie herunterzuladen: oben der Dateiname, die Zahl der Zeilen und der Link "als CSV herunterladen", darunter die Tabelle mit der ersten Zeile als Kopf.

Frage	Antwort
Trennzeichen	;, wenn die erste Zeile mehr davon hat als ,, sonst ,
Kodierung	UTF-8, wenn die Bytes gültiges UTF-8 sind, sonst cp1252 (Excel unter Windows)
Länge	höchstens 2000 Zeilen, darunter ein Hinweis auf die Gesamtzahl
die Datei selbst	datei.csv?raw=1

Der Inhalt steht auch in der Suche (`typ:csv`). Die älteren Stile zeigen dieselbe Tabellenseite in ihrem Aussehen. `--csv-view 0` schaltet die Tabellenseite ab; dann kommt die Datei unverändert.

5.8 Office-Dateien als Seite

Mit `--office-view auto` (Vorgabe) zeigt ein Klick auf eine Office-Datei eine Seite statt eines Downloads, für jedes Format, dessen Modul aus tclutils ladbar ist:

Datei	Seite	Modul
.docx	Absätze	tudocx
.odt, .odp	Absätze	tuodf
.xlsx	eine Tabelle je Blatt, höchstens 2000 Zeilen	tuxlsx
.ods	eine Tabelle je Blatt, höchstens 2000 Zeilen	tuodf

Oben stehen die Art, die Zahl der Absätze oder Blätter und der Link "herunterladen" auf `datei.docx?raw=1`. Eine Datei, die sich nicht lesen lässt, zeigt die Meldung und denselben Link. Die Startmeldung nennt die Formate:

```
  Ansicht: .docx .odt .odp .ods .xlsx als Seite
```

Zwischenablage. Eine große Tabelle zu lesen ist teuer: eine `.xlsx` mit 20 000 Zeilen kostete 4,5 s, bei jedem Aufruf, und in der Zeit wartete jede andere Anfrage. Darum bleibt eine fertige Office- oder CSV-Seite liegen, bis sich Änderungszeit oder Größe der Datei ändern. Ab dem zweiten Aufruf dauert dieselbe Seite 10 ms, auch in einem anderen Theme oder Stil. `--viewcache` setzt die Obergrenze in MB (Vorgabe 32); ist sie erreicht, fällt die am längsten nicht gebrauchte Seite heraus. `--viewcache 0` schaltet die Ablage ab.

Die Änderungszeit zählt in Sekunden. Wer eine Datei in derselben Sekunde auf genau dieselbe Größe ändert, sieht bis zur nächsten Änderung den alten Stand.

5.9 Suche

In der Kopfleiste steht ein Suchfeld. Es schickt `/?q=Wörter`. Die Adresse geht auch von Hand:

```
http://127.0.0.1:8080/?q=control+port
```

Regeln:

- Alle Wörter müssen vorkommen (UND). Wörter unter zwei Zeichen fallen weg.
- Groß und klein egal.
- Umlautfaltung: `ae`, `oe`, `ue`, `ss` treffen ä, ö, ü, ß — und umgekehrt. `groesse` findet "Größe".
- Gesucht wird im Text ohne Markdown-Zeichen und ohne Link-Ziele.
- Treffer im Titel (erste H1) zählen mehr als Treffer im Text.
- Die besten zuerst, 50 je Seite; ab 51 Treffern steht "1–50 von 213" da, dazu *zurück* und *weiter* (`seite=N`).

Suchsyntax über einfache Wörter hinaus:

Eingabe	**Wirkung**
"die Freigabe"	Wortfolge: die Wörter in dieser Reihenfolge, auch über einen Zeilenumbruch
-proxy, -"im Archiv"	Treffer mit diesem Wort oder dieser Folge fallen weg
a OR b, a ODER b, a | b	eins von beiden; bindet nur die beiden Nachbarn
in:buch/kapitel	nur unterhalb dieses Ordners, relativ zur Wurzel
typ:md, typ:pdf, typ:office, typ:csv	nur diese Art von Datei

`OR` und `ODER` gelten nur großgeschrieben; `or` ist ein gewöhnliches Suchwort. `in` und `typ` gehen auch als URL-Parameter (`/?q=router&in=netz&typ=pdf`); steht beides da, gilt die Angabe in der Anfrage. `...` und Punkt-Ordner in `in` zählen als "überall".

Eingrenzen geht auch ohne Syntax. Unter dem Suchfeld der Ergebnisseite stehen zwei Auswahlen: *Bereich* und *Typ*. Das Suchfeld in der Kopfleiste schickt den Ordner der aktuellen Seite als `von=` mit. *Bereich* bietet dann diesen Ordner und alle darüber an, jeweils mit ihrem Titel. Gesucht wird trotzdem zuerst überall; eingegrenzt wird erst, wenn man einen Bereich wählt. Aus einem Buchkapitel heraus heißt das: ein Klick, und die Suche bleibt im Buch.

Hilfe auf der Ergebnisseite gibt es für Leser, die die Syntax nicht kennen. Alles geht ohne JavaScript:

Teil	**Was er zeigt**
Suchtipps	aufklappbare Tabelle Eingabe, Wirkung, Beispiel. Die Beispiele sind Links aus dem eigenen Bestand: ein häufiges Titelwort, ein vorhandener Ordner für in:, nur die Typen, die an sind. Auf der leeren Suche (/?q=) offen
Gesucht:	die verstandene Anfrage in Bausteinen (Wort, Wortfolge, ohne, ODER-Paar, Bereich, Typ); das × daneben sucht ohne diesen Baustein

Teil	Was er zeigt
Nichts gefunden	"Überall suchen" mit Trefferzahl, wenn Bereich oder Typ gesetzt ist; die Zahl ohne die Minuswörter und mit den Wortfolgen als einzelnen Wörtern; "Meinten Sie" mit bis zu drei ähnlichen Wörtern aus dem Bestand; ein Hinweis, wenn ein klein geschriebenes or oder oder zwischen Wörtern steht
Zähler	Treffer je Typ und die drei Ordner mit den meisten Treffern, als Links

"Meinten Sie" nimmt Wörter mit höchstens zwei Abweichungen (Editierabstand), gefaltet wie die Suche: `fenstr` schlägt `fenster` vor.

In der Kopfleiste steht im Suchfeld `Suchen: "Wortfolge" -ohne`, daneben führt? zu den Suchtipps. Die Taste `/` springt ins Suchfeld; das ist das einzige kleine Skript, und es schweigt, wenn schon ein Eingabefeld den Fokus hat. Beim Tippen schlägt das Feld bis zu 200 Dokumenttitel vor, höhere Ebenen zuerst. Das ist ein `<datalist>` aus dem Suchindex, ohne Skript.

Jeder Treffer zeigt den Titel als Link, darunter den Ordnerpfad (`Bücher › Beispiel`, Ordnertitel wie in den Brotkrumen; bei einer Datei in der Wurzel leer) und einen Ausschnitt von etwa 220 Zeichen. Er beginnt kurz vor dem ersten Treffer im Text, auch wenn der weit hinten steht. Die Suchwörter sind darin markiert, auch in gemischter Schreibung: `groesse` markiert "Grösse", "Größe" und "GROESSE".

Bestand. Durchsucht werden alle `.md` unter `--root`. Jeder Text wird einmal gelesen und mit seiner mtime behalten; eine neue Suche liest nur geänderte Dateien neu. Punkt-Ordner bleiben draußen (außer mit `--dotfiles 1`), Symlinks immer — wie beim Ausliefern.

Den Suchindex baut mdserver gleich nach dem Start, im Hintergrund: in Schritten von höchstens 50 ms, dazwischen bedient er andere Anfragen. Bei 30 000 Dateien dauert das etwa 45 s, und Seiten kommen in dieser Zeit nach etwa 10 ms. Eine Suche wartet nicht darauf. Sie antwortet mit dem, was schon gelesen ist, und sagt es:

```
Der Suchindex wird nach dem Start noch aufgebaut; 22795 Dateien sind noch
nicht gelesen und fehlen in dieser Liste. Später noch einmal suchen.
```

Fehlen höchstens 500 Dateien, liest die Suche den Rest selbst; bei einem kleinen Bestand merkt man vom Aufbau also nichts. Die Titelvorschläge im Suchfeld erscheinen, sobald der Index steht. Mit `--pdf-worker 0` gibt es keinen Aufbau im Hintergrund; dann liest die erste Suche alles, auch die PDFs.

PDFs. Ist tclutils::tupdftext ladbar (ab tclutils 0.67.0, auf dem Modulpfad oder über TCLUTILS_TM), durchsucht?q= auch jede `.pdf` unter der Wurzel, Seite für Seite. Das ist die Vorgabe `--pdf-search auto`; `--pdf-search 1` warnt beim Start, wenn das Paket fehlt, `0` schaltet es ab. Ein Treffer heißt `handbuch, S. 17` und verlinkt auf `handbuch.pdf#page=17`. Firefox (pdf.js) und Chrome springen damit auf die Seite; andere Betrachter öffnen die Datei vorn.

Den Text liest `tupdftext` in reinem Tcl. Er landet in `--pdf-cache` (Vorgabe `$HOME/.cache/mdserver-pdf`, nie in der Wurzel), eine Datei je PDF, gültig solange mtime und Größe stimmen. Eine PDF ohne Text (Scan, verschlüsselt) gibt keinen Treffer. Kann `tupdftext` eine Datei nicht lesen (beschädigt, keine PDF), steht sie ebenfalls leer im Cache,

und das Protokoll sagt warum:

```
[10:29:26] pdf worker: /srv/md/pdfs/kaputt.pdf has no text, reader failed: not a
    PDF (no %PDF- header): /srv/md/pdfs/kaputt.pdf
```

Gelesen wird in einem eigenen `tclsh` je PDF, dem PDF-Worker (`lib/pdftext-worker.tcl`, `--pdf-worker 1`, Vorgabe). mdserver startet ihn mit `tclutils::tutask`, das ohnehin neben `tupdftext` liegt:

```
await [pipe [list $tclsh pdftext-worker.tcl $pdf ...] -encoding utf-8] \
    -timeout 600000
```

`await` hält nur die Aufgabe an, die die PDFs der Reihe nach abarbeitet; der Ereignis-Loop läuft weiter. Der Cache füllt sich gleich nach dem Start und danach für jede neue oder geänderte PDF. mdserver wartet nicht auf ihn: Eine Suche liefert, was im Cache steht, und sagt dazu "3 PDFs werden noch gelesen; ihre Seiten fehlen in dieser Liste". Seiten und alle anderen Anfragen kommen sofort.

Mit `--pdf-worker 0` liest mdserver die PDFs im eigenen Prozess. mdserver hat einen einzigen Ereignis-Loop, und solange `tupdftext` läuft, steht er für alle. Gemessen mit 16 PDFs (1804 Seiten), leerer Cache:

	erste Suche	Seite, gleichzeitig angefragt
im Serverprozess (--pdf-worker 0)	36,4 s	36,3 s
mit Worker	0,03 s	0,04 s

Der Worker brauchte für die 16 PDFs 35,9 s, so lange wie im Serverprozess: ein `tclsh` je PDF kostet keine messbare Zeit; danach dauert eine Suche über alle Seiten 0,03 s. Nach einem Neustart mit gefülltem Cache 0,3 s für die erste Suche, weil die Cache-Dateien eingelesen werden.

Wenn es keinen Worker gibt oder er scheitert:

- Ohne `tclutils::tutask`, ohne `pdftext-worker.tcl` oder wenn mdserver als Starpack läuft (`info nameofexecutable` startet dann die Anwendung, keinen `tclsh`), liest mdserver selbst, wie mit `--pdf-worker 0`.
- Scheitert der Worker an einer PDF (Absturz, Exit-Code, mehr als 10 Minuten), steht es im Protokoll, und die nächste Suche versucht es noch einmal. Beim zweiten Mal bleibt die PDF draußen, bis sie sich ändert (leerer Eintrag im Cache).
- Hält mdserver über den Steuerport an (`mdctl.tcl … stop`), bricht `tutask` den laufenden Worker ab. Stirbt mdserver hart, liest der Worker seine eine PDF zu Ende und endet dann.

Ob die PDF-Suche an ist, steht beim Start im Banner:

```
  PDF:   Suche an (tclutils::tupdftext 0.1, Cache /var/cache/mdserver-pdf)
  PDF:   Suche aus (--pdf-search 0)
  PDF:   Suche aus: tclutils::tupdftext nicht gefunden (TCLUTILS_TM setzen oder
    tclutils/lib/tm auf den Modulpfad)
```

Die erste Zeile gehört zu einem Start mit `--pdf-cache /var/cache/mdserver-pdf`.

Außer im Banner meldet `auto` nichts. Unter systemd gehört `TCLUTILS_TM` deshalb in die Unit (`Environment=TCLUTILS_TM=…`).

Office-Dateien. Mit `--office-search auto` (Vorgabe) durchsucht?q= auch Office-Dateien,

jedes Format, dessen Modul aus tclutils ladbar ist: `.odt`, `.ods`, `.odp` mit `tuodf`, `.docx` mit `tudocx`, `.xlsx` mit `tuxlsx`. Ein Treffer je Datei, verlinkt auf die Datei selbst. Gelesen wird der Text der Absätze, auch in Tabellen, Listen und auf Folien, bei Tabellenkalkulationen der Inhalt aller Zellen aller Blätter. Überschriften in ODF findet erst `tuodf` 0.2. Das Lesen geht schnell (ein ZIP und wenige XML-Teile) und läuft im Serverprozess, ohne Worker; der Text bleibt mit der mtime im Suchindex. Das Banner nennt die Formate:

```
  Office: Suche an (.odt .ods .odp .docx .xlsx; tuodf 0.2, tudocx 0.1, tuxlsx 0.1)
```

Suche als JSON. `/?q=…&format=json` gibt die Treffer als JSON zurück, in jedem Stil, mit `in` und `typ` wie oben. Das braucht `tclutils::tujson`; ohne es antwortet mdserver mit **501**.

```
{"query":"proxy_pass","pending":0,"indexing":0,"total":1,"page":1,"pages":1,
 "hits":[{"url":"/a/eins.md","title":"Netzwerk","path":"A",
 "snippet":"… der proxy_pass Wert.","score":1}]}
```

`snippet` ist reiner Text ohne `<mark>`. `pending` zählt die PDFs, die der Worker noch liest, `indexing` die Dateien, die der Index nach dem Start noch nicht gelesen hat. `total` zählt alle Treffer, `hits` sind die 50 der Seite `page` (`&seite=N`). So kann ein Skript oder ein Editor die Suche benutzen, ohne HTML zu zerlegen.

Ältere Stile. `?q=` geht auch dort. Die Antwort ist eine schlichte Liste "Suche: …" mit Titel und Ordnerpfad, ohne Ausschnitt, dazu *Gesucht:*, die Hilfe bei *Nichts gefunden* und die Seiten. Ein Suchfeld gibt es dort nicht.

Suche und Ansicht als Module. Der Code der Suche liegt in `lib/mdserver/search-0.4.tm` (Klasse `mdserver::Search`), die Office- und CSV-Seiten in `lib/mdserver/view-0.4.tm` (`mdserver::View`). Beide sind Mixins der Server-Klasse: ihre Methoden laufen im Server-Objekt und teilen dessen Variablen. Allein sind sie nicht zu gebrauchen; bei einer Installation müssen die Dateien neben `mdserver-0.4.tm` liegen (Kapitel *Starten*).

5.10 Zusammenfassung

- `site` ist ein ganzes Layout, ohne JavaScript, mit einem Stylesheet.
- Ordner ohne `index.md` werden Kacheln: Bücher, Bereiche, Dokumente, Dateien.
- Dunkel nach System oder `theme=dunkel`; Menüknopf unter 820 px.
- Handbuchseiten mit `# NAME` heißen nach dem Befehl.
- `/?q=` sucht UND-verknüpft, ohne Groß/klein, mit Umlautfaltung, 50 Treffer je Seite.
- Suchtipps, *Gesucht:*, Hilfe bei *Nichts gefunden* mit "Meinten Sie".
- Der Suchindex entsteht nach dem Start im Hintergrund; eine Suche in dieser Zeit sagt, wie viel noch fehlt.
- Mit tclutils (ab 0.67.0) auch in PDFs, ein Treffer je Seite (`datei.pdf#page=N`); der Text liegt im Cache außerhalb der Wurzel.
- Wortfolgen in Anführungszeichen, `-wort`, `OR`, `in:` und `typ:`; *Bereich* und *Typ* auf der Ergebnisseite.
- Mit tclutils 0.69.0 auch Office-Dateien (ODF, DOCX, XLSX) in Suche und Ansicht, CSV als Tabelle und die Suche als JSON (`format=json`).
- Office- und CSV-Seiten bleiben liegen, bis sich die Datei ändert (`--viewcache`).

Nächstes Kapitel: HTTP-Verhalten

6 HTTP-Verhalten

6.1 Einführung

Nur `GET` und `HEAD`. Andere gültige Methoden: **405** mit `Allow: GET, HEAD`. `HEAD` hat denselben Status und dieselbe `Content-Length` wie `GET`, aber keinen Körper — auch bei 206 und 404.

Der Kopf `Server:` kommt aus `package provide mdserver`, nicht aus einer zweiten Zahl im Skript.

6.2 Routing

URL	Verhalten
/	index.md oder Verzeichnisseite
/datei.md	Markdown → HTML
/datei	Clean URL: /datei.md
/verzeichnis	301 auf /verzeichnis/
/verzeichnis/	index.md, sonst Buch-Startseite (mit book.tcl), sonst Verzeichnisseite
/buch/index.md ohne Datei	Buch-Startseite, wenn book.tcl daneben liegt
/?q=Wörter	Volltextsuche
/?q=Wörter&format=json	Suche als JSON
/tabelle.csv, /brief.docx …	Tabellen- bzw. Textseite (--csv-view, --office-view)
/tabelle.csv?raw=1	die Datei selbst
/?nav=index	alle Dokumente als Baum
/__mdserver/health	ok oder 503
--statuspath	Status als Text, nur wenn eingeschaltet
.css .js Bilder .pdf	Datei
Pfad mit Punkt-Bestandteil	403 bei --dotfiles 0
unbekannt	404

Eine Clean URL gilt in jedem Stil und in Unterordnern: `/tcltk/dict` liefert `tcltk/dict.md`. Gedacht für `nroff2md` (Kapitel *Site aufbauen*). Ein 301 auf einen Ordner behält den Query-String.

Verzeichnisseite ohne `index.md`: im Stil `site` Kacheln (Bücher, Bereiche, Dokumente, Dateien), gebaut von mdserver selbst. In den älteren Stilen eine Liste mit Unterordnern, Markdown-Titeln aus H1 und Link nach oben, gerendert über mdstack — mit denselben`?style=` -Parametern.

6.3 Dateien

- **Range / 206:** `bytes=0-99`, `bytes=1000-`, `bytes=-50`. Große PDFs im Browser springbar.
- **416** nur, wenn der Bereich nicht erfüllbar ist: Anfang hinter dem Dateiende oder Anfang nach Ende. Ein unlesbarer `Range`-Kopf (`bytes=abc`) wird übergangen: 200 mit dem ganzen Inhalt.
- **If-Range:** passt der Validator nicht, kommt 200 mit dem ganzen Inhalt (RFC 9110). Sonst klebt ein Viewer Stücke zweier Fassungen.
- **If-Modified-Since / 304:** unveränderte Dateien nicht neu senden.

6.4 Grenzen

- Anfragezeile länger als `--maxline`: **414**.
- Eine Kopfzeile länger als `--maxline` oder alle Köpfe zusammen länger als `--maxheader`: **431**.
- Danach bedient der Server normal weiter.

Dazu kommen Verbindungsgrenzen. mdserver hält je offener Verbindung eine Coroutine, bis die Anfrage da ist oder das Lese-Timeout (15 s) greift. Wer viele Verbindungen öffnet und nichts schickt, würde den einen Thread sonst allein beschäftigen.

Option	Vorgabe	darüber
--maxconn	200 insgesamt	503 mit Retry-After: 1
--maxperaddr	32 je Absenderadresse	503 mit Retry-After: 1

Über der Grenze antwortet mdserver, ohne die Anfrage zu lesen, und schließt. Das Protokoll nennt Adresse und Grund:

```
[09:20:11] 503 127.0.0.1 -- zu viele Verbindungen von 127.0.0.1 (32)
```

Hinter einem Proxy kommen alle Anfragen von dessen Adresse. Darum zählen Adressen aus `--trusted-proxy` nicht je Absender, nur insgesamt.

6.5 Antwortzeiten

Viele kleine Anfragen gleichzeitig sind kein Problem. Eine teure Anfrage aber hält alle anderen auf, bis sie fertig ist; es gibt nur einen Thread. Die Zahlen stammen von einem Rechner mit zwei Kernen, Tcl 8.6, Markdown-Dateien zu je etwa 5 KB:

Lage	3 000 Dateien	30 000 Dateien
kleine Seite, ruhig	5 ms	5 ms
20 Klienten gleichzeitig, kleine Seite	0,1 s für alle	0,1 s für alle
Suchindex nach dem Start (im Hintergrund)	4–8 s	etwa 45 s
kleine Seite, während der Index entsteht	10 ms	10 ms
Suche	0,1 s	1 s
Suche ohne Treffer, mit "Meinten Sie"	0,1 s	1 s

Lage	3 000 Dateien	30 000 Dateien
Alle Dokumente	0,1–0,2 s	0,8 s

Unter Tcl 9.0 liegen die Werte in derselben Größenordnung. Eine `.xlsx` mit 20 000 Zeilen als Seite kostet beim ersten Aufruf 4,5–7 s, danach 10 ms (`--viewcache`, Kapitel *Seitenlayout und Suche*).

6.6 Protokoll

Eine Zeile je Anfrage, nach jeder Zeile Flush (unter systemd sonst erst Kilobytes später sichtbar). Adresse = Gegenstelle, oder der erste `X-Forwarded-For`, wenn die Gegenstelle in `--trusted-proxy` steht.

```
[09:15:03] 127.0.0.1 GET /index.md 200 6194 markdown
[09:15:04] 127.0.0.1 GET / 200 8120 search
[09:15:05] 127.0.0.1 GET /buch/ 200 7311 book landing
[09:15:06] 127.0.0.1 GET /tcltk 301 0 -> /tcltk/
[09:15:07] 127.0.0.1 GET /.env 403 89 hidden files are not served: /.env
[09:15:08] 127.0.0.1 GET /pdfs/handbuch.pdf 206 100 bytes 0-99/306855
[09:15:09] 10.1.2.3 GET / 200 13247 index.md
```

Die Notiz am Ende sagt, welcher Weg genommen wurde:

Notiz	Weg
markdown, index.md, book landing	Markdown-Seite, index.md eines Ordners, erzeugte Buch-Startseite
directory index, site index, search	Verzeichnisseite, Alle Dokumente, Suche
view csv, view xlsx …	Tabellen-oder Textseite einer CSV-oder Office-Datei
MIME-Typ (application/pdf)	ganze Datei, auch nach einem If-Range, der nicht passt
bytes A-B/GRÖSSE, range	206, 416
not modified	304
-> /ziel/	301
Fehlertext	403 (Punkt-Datei, Symlink hinaus) und 500

Die letzte Beispielzeile kam über einen Proxy aus `--trusted-proxy`: Sie nennt die Adresse aus `X-Forwarded-For`.

Der Pfad steht ohne Query-String da; eine Suche erkennt man an der Notiz `search`.

`--no-log` schaltet ab.

6.7 Zusammenfassung

- GET und HEAD. Alles andere 405 oder gar keine HTTP-Antwort.
- Clean URLs ohne `.md`, auch in Unterordnern.

- Range und 304 gelten für Dateien, nicht nur für Markdown.
- 416 nur bei unerfüllbarem Bereich; kaputter Range-Kopf gibt 200.
- Zu viele offene Verbindungen: sofort 503 (`--maxconn`, `--maxperaddr`).
- Eine teure Anfrage hält die anderen auf; die Suche bei 30 000 Dateien etwa 1 s.

Nächstes Kapitel: Control-Port

7 Control-Port

7.1 Einführung

`--control` PORT öffnet einen Steuerkanal **nur auf localhost**. Darüber endet der Prozess sauber — ohne `fuser -k`, ohne PID-Suche. Er sagt auch, wie es dem Dienst geht (`status`), und leert die Zwischenspeicher (`reload`).

7.2 Bedienen

```
tclsh mdserver.tcl --root /srv/md --control 8099
tclsh mdctl.tcl --port 8099 ping     # -> pong
tclsh mdctl.tcl --port 8099 status   # mehrere Zeilen
tclsh mdctl.tcl --port 8099 reload   # -> reloading
tclsh mdctl.tcl --port 8099 stop     # -> stopping
```

`mdctl.tcl` liegt neben `mdserver.tcl`.

Option	Standard	Bedeutung
--port	—	Steuerport (--control des Servers), Pflicht
--host	127.0.0.1	Adresse des Steuerports
--timeout	10000	Millisekunden, die mdctl.tcl auf die Antwort wartet

Rückgabewert 0, wenn eine Antwort kam, sonst 1 (Meldungen im Anhang *Fehlermeldungen*). systemd braucht die 0.

Kein `echo stop | nc`. OpenBSD-netcat auf Debian schließt die Senderichtung nach EOF nicht; ohne `-N` läuft die Verbindung in den Timeout. Andere `nc`-Varianten kennen `-N` nicht. Tcl ist da, wo mdserver läuft — deshalb `mdctl.tcl`.

7.3 Status

`status` sagt, was der Dienst seit dem Start getan hat:

```
mdserver: 0.4, Tcl 8.6.14
started: … 10:37:35 (up 2 h 14 min)
root: /srv/md
http: 127.0.0.1:8080
connections: 2 open, peak 17, limits 200 / 32 per address, rejected 0
requests: 4211 (2xx 4102, 3xx 31, 4xx 78, 5xx 0)
search index: 3002 files, checked 12 s ago
pdf queue: 0
view cache: 3 pages, 0.9 of 32 MB, 40 hits, 3 built
```

```
slowest:
    4728 ms  11-05 10:37 GET /tabellen/gross.xlsx 200 view xlsx
     612 ms  11-05 11:02 GET / 200 search
```

Zeile	Inhalt
started	Startzeit und Laufzeit
http, https	Adresse, an die gebunden wurde
connections	offen, höchstens gleichzeitig, Grenzen, mit 503 abgewiesen
requests	Anfragen nach Statusklasse
search index	Dateien im Index; während des Aufbaus building (1234 of 30002 files read)
pdf queue	PDFs, die der Worker noch liest
view cache	Office-und CSV-Seiten in der Zwischenablage, Treffer und neu gebaute
slowest	die fünf langsamsten Anfragen: Dauer, Zeit, Pfad ohne Query, Status, Notiz der Protokollzeile

`view cache` ist die Zwischenablage für Office- und CSV-Seiten (`--viewcache`). Die Zähler liegen im Speicher; nach einem Neustart beginnen sie bei null. `slowest` zeigt, wo der eine Thread hängen bleibt: hier eine große Tabelle beim ersten Aufruf.

Dasselbe gibt es über HTTP, wenn man es einschaltet:

```
tclsh mdserver.tcl --root /srv/md --statuspath /__mdserver/status
```

Die Vorgabe ist aus. Die Seite nennt Pfade und Zähler; sie gehört hinter eine Anmeldung.

7.4 Reload

`reload` vergisst die Office- und CSV-Seiten der Zwischenablage und die Titel der Listen und liest den Suchindex im Hintergrund neu. Bis der neue steht, antwortet der alte.

Nötig ist das selten. Suche und Zwischenablage erkennen eine geänderte Datei an Änderungszeit und Größe. `reload` ist für den Fall, dass beides gleich bleibt, etwa ein Ordner, der mit alten Zeitstempeln zurückgespielt wurde.

7.5 Was stop tut

1. Horchsockets zu: keine neue Verbindung.
2. Laufende Antworten: **5 s** Zeit.
3. Danach kappen, Protokollzeile `shutdown: 1 Verbindung(en) abgeschnitten`.

Werden alle Antworten rechtzeitig fertig, steht im Protokoll `shutdown: fertig`.

Ohne diesen Ablauf schneidet ein Restart eine große PDF mitten im Byte ab.

7.6 systemd (Kennwerte, nicht der Installationsweg)

```
ExecStart=... mdserver.tcl --root /srv/md --port 8080 --bind 127.0.0.1 \
    --trusted-proxy 127.0.0.1 --control 8099
ExecStop=... mdctl.tcl --port 8099 stop
TimeoutStopSec=15
```

`TimeoutStopSec` über den 5 s. Ohne `ExecStop` kommt SIGTERM. Tcl ohne TclX hat keine Signalbehandlung — der Prozess endet mitten in der Antwort.

Unit-Datei als Beispiel: `mdserver.service.beispiel` neben dem Skript. Container, nginx, Samba: LAN-Bücher.

7.7 Zusammenfassung

- Control-Port = localhost, `ping`, `status`, `reload`, `stop`.
- `status` zeigt Verbindungen, Anfragen, Suchindex und die langsamsten Anfragen; `--statuspath` dasselbe über HTTP, Vorgabe aus.
- Stop wartet 5 s auf laufende Antworten.
- ExecStop gehört zur Unit, nicht SIGTERM allein.

Nächstes Kapitel: TLS

8 TLS

8.1 Einführung

HTTPS ist optional. Paket `tls`, Dateien `--cert` und `--key`, Port `--tlsport` (Vorgabe 8443). Für das LAN hinter nginx oft unnötig: TLS endet am Proxy, mdserver bleibt HTTP auf Loopback.

Fehlt das Paket `tls`, bricht der Start nicht ab. Es kommt eine Warnung, HTTP läuft weiter, HTTPS nicht.

8.2 Zertifikat bauen

`mkcert.tcl` neben dem Server. SAN muss stimmen: wer `https://192.168.1.50/` aufruft, braucht die **Adresse** im SAN, nicht nur den Namen. `--san` hängt weitere Namen oder Adressen an, mehrfach angebbar; der CN steht immer drin:

```
tclsh mkcert.tcl --cn mdstack --san 192.168.1.50
tclsh mkcert.tcl --cn mdstack --san 192.168.1.50 --san mdstack.lan
```

Das Skript liest den SAN nach dem Erzeugen zurück und bricht ab, wenn er fehlt.

Vorhandene Zertifikate ohne SAN werden ersetzt — Browser lehnen sie ab (`ERR_CERT_COMMON_NAME_INVALID`), auch nach Import.

Die OpenSSL-Option `-addext` (ein Strich) braucht OpenSSL ab 1.1.1. Fehlt sie, bricht `mkcert.tcl` ab, statt still ohne SAN zu bauen. `--no-san` erzwingt das alte Verhalten ausdrücklich.

8.3 Dateien

`mkcert.tcl` schreibt `server.crt` und `server.key` nach `--out`. Vorgabe ist das **aktuelle Verzeichnis**, nicht das Skriptverzeichnis. Darum vorher in den Zielordner wechseln oder `--out` angeben:

```
tclsh mkcert.tcl --cn mdstack --san 192.168.1.50 --out /etc/mdserver
tclsh mdserver.tcl --root /srv/md \
    --cert /etc/mdserver/server.crt --key /etc/mdserver/server.key
```

`server.crt` und `server.key` gehören nicht ins Git. Selbstsigniert: Browser-Warnung, nur Entwicklung. Öffentlich: Let's Encrypt am Proxy, nicht in mdserver.

Der Schlüssel liegt **nie** unter `--root`. mdserver liefert jede Datei der Wurzel aus, deren Name nicht mit einem Punkt beginnt: ein `server.key` in `/srv/md/` holt jeder mit `GET /server.key`.

8.4 Zusammenfassung

- TLS in mdserver ist die Ausnahme, Proxy die Regel.
- SAN prüfen, nicht nur das Datum; Adressen mit `--san`.
- `mkcert.tcl` schreibt ins aktuelle Verzeichnis oder nach `--out`.
- Schlüssel außerhalb von `--root`, nie im Buch- oder Dokumentenbaum.

Nächstes Kapitel: Hinter dem Proxy

9 Hinter dem Proxy

9.1 Einführung

Öffentlich oder im LAN mit Anmeldung: Reverse-Proxy davor, mdserver auf Loopback. Dieses Kapitel ist der Vertrag des Dienstes. nginx-Konfiguration, LXC und Firewall stehen in *Nginx* und den LAN-Büchern.

9.2 Bind und Weiterleitung

```
tclsh mdserver.tcl --root /srv/md --bind 127.0.0.1 --trusted-proxy 127.0.0.1
```

`--bind 127.0.0.1` ist die Vorgabe; es steht hier, damit die Zeile nichts dem Zufall überlässt. `X-Forwarded-For` wird **nur** gelesen, wenn die Gegenstelle in `--trusted-proxy` steht. Sonst schreibt jeder, der den Port erreicht, eine beliebige Adresse ins Protokoll.

Nur der **erste** Eintrag des Kopfes zählt. Alles dahinter kann der Klient selbst gesetzt haben.

Ohne `--trusted-proxy` wird der Kopf nicht gelesen.

In nginx:

```
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

9.3 Pfade und Symlinks

Directory Traversal ist gesperrt. Jeder Pfadbestandteil wird aufgelöst und erneut gegen die Wurzel geprüft: ein Link aus der Wurzel hinaus ist **403**. `--root /srv/md` passt nicht auf

`/srv/mdxyz`.

Dasselbe gilt für alles, was mdserver auflistet: Kopfleiste, Seitenleiste, Kacheln und *Alle Dokumente* zeigen von einem Link aus der Wurzel hinaus weder Namen noch Titel. Ein Ordner-Link auf einen eigenen Elternordner wird in diesen Listen übersprungen, statt im Kreis zu laufen. Ist `--root` selbst ein Link, gilt sein Ziel als Wurzel.

Versteckte Dateien: `--dotfiles` (Vorgabe aus). Jeder Pfad mit einem Bestandteil, der mit einem Punkt beginnt (`/.git/HEAD`, `/a/.env`), ist dann ebenfalls **403**. Die Suche lässt dieselben Ordner aus und folgt keinem Symlink.

9.4 Methoden und Grenzen

POST und andere Schreibmethoden: 405. Zeilen- und Kopfgrenzen: `--maxline`, `--maxheader`. Slow-loris: Lese-Timeout, Coroutine pro Verbindung, dazu `--maxconn` (200) und `--maxperaddr` (32). Hinter dem Proxy zählt `--maxperaddr` für dessen Adresse nicht, sofern sie in `--trusted-proxy` steht; sonst wären nach 32 gleichzeitigen Nutzern alle weiteren abgewiesen (Kapitel *HTTP-Verhalten*).

9.5 Zusammenfassung

- Loopback plus trusted-proxy, sonst lügt das Protokoll.
- 403 bei Symlink hinaus und bei Punkt-Dateien, nicht 200 mit `/etc/passwd` oder `/.git/config`.
- Proxy-Setup: Nachbarbücher, nicht hier noch einmal.

Nächstes Kapitel: Site aufbauen

10 Site aufbauen

10.1 Einführung

Von einem leeren Ordner zu einer lesbaren Site. Inhalte und Server getrennt halten: die Wurzel enthält nur, was ausgeliefert werden darf.

10.2 Mindestbaum aus dem Repo

```
arbeit/
  mdstack/
    lib/
      mdstack/parser-0.9.2.tm
      mdstack/html-0.2.tm
      mdstack/theme-0.1.tm
    tools/
      mdserver/
        lib/mdserver-0.4.tm
        lib/mdserver/search-0.4.tm
        lib/mdserver/view-0.4.tm
        lib/pdftext-worker.tcl
        styles/
```

```
      mdserver.tcl
      mdctl.tcl
      mkcert.tcl
  docir/
    lib/tm/
      docir-0.1.1.tm
      docir/...
  meinprojekt/
    docs/
      index.md
      einleitung.md
      api/
        index.md
```

`docs/` ist `--root`. `tools/mdserver/` ist der Dienst. Ohne das mdstack-`lib/` und ohne `docir/` daneben startet er nicht. Der Launcher findet beide selbst (Kapitel *Starten*). Für eine Installation ohne Repo gehören die Module in `lib/` neben `mdserver.tcl` — die genaue Liste steht ebenfalls dort.

10.3 Start

```
cd arbeit/mdstack/tools/mdserver
tclsh mdserver.tcl --root ../../../meinprojekt/docs --port 8080 \
    --title "Mein Projekt" --control 8099
```

`http://127.0.0.1:8080/`. Ohne `index.md` erscheinen die Kacheln.

Bücher brauchen keinen Vorlauf mehr. Fehlt die `index.md`, erzeugt mdserver die Startseite aus `book.tcl`. `book-webindex.tcl` (bookkit) ist optional — für eine feste `index.md` und ein `stichwortverzeichnis.md`. `mdstack::indexgen` schreibt bei Bedarf `index.md` für Ordner ohne `book.tcl`; im Stil `site` reichen die Kacheln.

10.4 Empfohlene Site

Eine Wurzel für das LAN, Dienst und Schlüssel daneben:

```
/srv/md/                          --root
  index.md                        Startseite
  buecher/
    book-beispiel/
      README.md
      Beispiel-de/
        book.tcl
        010-einleitung.md
        ...
      Beispiel-de.pdf
      Beispiel-de.html
    book-zweites/
      ...
  tcltk/
    index.md                      aus nroff2md
    dict.md
    ...
```

```
  notizen/
    ...
  pdfs/
    handbuch.pdf                  mit tclutils durchsuchbar
/opt/mdserver/                    Dienst
  mdserver.tcl
  mdctl.tcl
  lib/
  styles/
/etc/mdserver/                    Schlüssel, außerhalb von --root
  server.crt
  server.key
```

Bücher. Die bookkit-Ordner `book-xyz/` kommen unverändert nach `buecher/`, samt den gebauten `.pdf` und `.html` neben dem Buchordner. `/buecher/` zeigt sie als Buch-Kacheln mit Untertitel und Sprache. Im Buch stehen PDF und HTML über der Kapitelliste und auf der Startseite.

Tcl/Tk-Handbuch. `nroff2md` (aus `mdstack/tools/`) wandelt die nroff-Manpages der Tcl/Tk-Quellen in Markdown. Für einen Unterordner gehört `--linkmode file` dazu:

```
tclsh nroff2md.tcl --batch tcltkdoc/ /srv/md/tcltk/ --linkmode file
```

`--linkmode server` schreibt Verweise als `/dict` — absolut zur Wurzel. Das passt nur, wenn das Handbuch selbst `--root` ist. In `/srv/md/tcltk/` zeigte `/dict` ins Leere. `--linkmode file` schreibt `dict.md`, relativ und damit in jedem Ordner richtig.

Schlüssel. `server.crt` und `server.key` liegen außerhalb von `--root`, etwa in `/etc/mdserver/`. Nie in `/srv/md/`: dort liefert mdserver sie aus. Hinter nginx braucht mdserver gar keinen Schlüssel.

10.5 Demo

Unter `tools/mdserver/demo/` liegen `start.tcl` und Beispieldocs in `docs/`, darunter ein Buch. Zum Ansehen des Dienstes, nicht als Produktionswurzel.

```
cd tools/mdserver
tclsh mdserver.tcl --root demo/docs --port 8080
```

10.6 Zusammenfassung

- Wurzel = Inhalte. Skriptbaum = Dienst. Schlüssel woanders.
- mdstack-`lib/` und docir gehören zum Dienst.
- `index.md` in der Wurzel ist die Startseite.
- Bücher brauchen keinen webindex-Lauf; bookkit-Ordner nach `buecher/`.
- Handbuch im Unterordner: `nroff2md --linkmode file`.
- PDFs in der Wurzel sind mit tclutils seitenweise durchsuchbar.

Nächstes Kapitel: Fallen

11 Fallen

11.1 Einführung

Betriebsfehler, die wie "der Server hängt" aussehen.

11.2 Nach dem Update nicht mehr im LAN erreichbar

Seit November 2026 horcht mdserver ohne `--bind` nur auf `127.0.0.1`. Wer ihn bisher ohne `--bind` für das ganze Netz gestartet hat, braucht jetzt `--bind '*'` (mit Anführungszeichen) oder die Adresse der Schnittstelle. Hinter nginx bleibt es bei `127.0.0.1`.

11.3 `--bind '*'` ohne Anmeldung

Der Port ist auf allen Schnittstellen offen, ohne Anmeldung. Die Startmeldung warnt ("Achtung: von anderen Rechnern erreichbar"). Nur im vertrauten Netz, sonst Proxy mit Anmeldung davor.

11.4 Hinter dem Proxy 503 bei vielen Nutzern

`--maxperaddr` (32) zählt je Absenderadresse. Hinter einem Proxy ist das immer dieselbe. Die Adresse des Proxys gehört in `--trusted-proxy`; dann zählt nur noch `--maxconn`.

11.5 Suche findet nach dem Start zu wenig

Der Suchindex entsteht nach dem Start im Hintergrund, bei großen Beständen eine Weile. Bis dahin sagt die Ergebnisseite, wie viele Dateien noch fehlen. Später noch einmal suchen.

11.6 `echo stop | nc`

Läuft in den Timeout. `mdctl.tcl` verwenden.

11.7 SIGTERM statt ExecStop

Große PDF wird abgeschnitten. Control-Port in der Unit.

11.8 `?style=sidebar` ändert nichts

CSS-Datei nicht gefunden. `--stylesdir` prüfen, Dateien neben `lib/` (`styles/sidebar.css`). Dasselbe gilt für `site`: ohne `styles/site.css` steht das Layout ungestylt untereinander.

11.9 `?theme=solarized` oder `?toc=0` ändert nichts

Der Vorgabestil ist `site`. Dort zählt nur `theme=dunkel`; Theme-Namen, `toc`, `--navbg`, `--navfg`, `--navmax` gelten für die älteren Stile. Abhilfe:`?style=plain&theme=solarized` oder `--style plain`.

11.10 Die Seiten sehen nach dem Update anders aus

Vorgabe ist `site`, früher war es `plain`. Wer das alte Aussehen will, startet mit `--style plain`.

11.11 `can't find package docir::mdSource`

mdstack allein reicht nicht. `mdstack::html` 0.2 braucht fünf docir-Module. Aus dem Repo: `docir/` neben `mdstack/` legen. Bei einer Installation: `docir-0.1.1.tm` und `docir/` in das `lib/` neben `mdserver.tcl` (Kapitel *Starten*).

11.12 `lib/mdserver/` fehlt

Wer nach dem Update nur `mdserver-0.4.tm` kopiert hat, bekommt beim Start `mdserver::search 0.4 not found` oder `mdserver::view 0.4 not found`. Der Ordner `lib/mdserver/` mit `search-0.4.tm` und `view-0.4.tm` gehört dazu.

11.13 Module flach in `lib/`

`html-0.2.tm` direkt in `lib/` statt in `lib/mdstack/` wird nicht gefunden. Der Unterordner ist Teil des Paketnamens.

11.14 `X-Forwarded-For` ohne trusted-proxy

Jeder Klient könnte sich die Protokolladresse selbst setzen. Darum liest mdserver den Kopf nur von Gegenstellen aus `--trusted-proxy`; ohne die Option steht im Protokoll die Gegenstelle, bei nginx davor also immer `127.0.0.1`.

11.15 Wurzel = Git-Arbeitskopie, `--dotfiles 1`

`/.git/config` geht raus. Vorgabe ist aus — so lassen.

11.16 `server.key` unter `--root`

mdserver liefert jede Datei der Wurzel, deren Name nicht mit einem Punkt beginnt. `GET /server.key` gibt den Schlüssel heraus. Schlüssel außerhalb der Wurzel ablegen.

11.17 Clean-URL-Link auf eine Datei ohne `.md`

404, wenn die Datei nur als `.md` existiert und der Link einen anderen Stamm hat. Ziel und Dateiname angleichen.

11.18 Tcl/Tk-Handbuch im Unterordner, Verweise ins Leere

Mit `nroff2md --linkmode server` erzeugt, zeigen die Verweise auf `/dict` — die Wurzel, nicht den Unterordner. Für einen Unterordner mit `--linkmode file` neu erzeugen.

11.19 Suche findet eine Datei nicht

Die Suche liest `.md` und, mit tclutils, `.pdf`, Office-Dateien und `.csv` (Banner-Zeilen `PDF:`, `Office:`, `CSV:`). Punkt-Ordner bleiben draußen (bei `--dotfiles 0`), Symlinks immer — auch solche, die innerhalb der Wurzel bleiben. Wörter unter zwei Zeichen werden ignoriert.

11.20 Suche findet keine PDF

Banner-Zeile `PDF:` lesen. `Suche aus: tclutils::tupdftext nicht gefunden` heißt: tclutils fehlt auf dem Modulpfad. Unter systemd gehört `Environment=TCLUTILS_TM=…` in die Unit, die Shell-Umgebung gilt dort nicht. Ist die Suche an, liest der Worker die PDFs nach dem Start nach und nach; bis dahin meldet die Trefferseite "… PDFs werden noch gelesen". Eine gescannte PDF ohne Textebene gibt nie einen Treffer.

11.21 `or` findet nichts mehr

`a or b` sucht drei Wörter, darunter `or`. Nur `OR`, `ODER` und `|` verbinden zwei Wörter zu "eins von beiden".

11.22 Eine CSV- oder Office-Datei lässt sich nicht herunterladen

Ein Link auf eine `.csv` oder eine Office-Datei öffnet die Tabellen- oder Textseite, in jedem Stil. Die Datei selbst liefert `?raw=1` (Link oben auf der Seite), für alle Dateien `--csv-view 0` bzw. `--office-view 0`.

11.23 Datum aus einer XLSX ist eine Zahl

Excel speichert ein Datum als Seriennummer. Die Suche findet "46325", nicht "30.10.2026".

11.24 Handbuchseiten heißen alle "NAME"

Nur in *Alle Dokumente* (`/?nav=index`). Seitenleiste, Kacheln, Titel und Suche nehmen `CommandName` oder die NAME-Zeile (Kapitel *Seitenlayout und Suche*).

11.25 Älterer Stand: Befehle über die URL

Bis zum Stand vom 30.10.2026 lief der URL-Pfad durch `subst`. Eine Anfrage auf `/[exec id]` führte `id` als Server-Benutzer aus, auch hinter einem Proxy. Jede ältere Installation aktualisieren; der neue Stand dekodiert nur `%XX` (Test `url-1`), ein kaputtes `%` im Pfad ist 400.

11.26 Dateinamen mit Umlauten: 404

Unter systemd ist `LANG` nicht gesetzt, Tcl liest Dateinamen dann als iso8859-1. Die Startmeldung warnt mit `Achtung: Systemkodierung`. In der Unit `Environment=LANG=C.UTF-8` setzen (steht in `mdserver.service.beispiel`).

Nächstes Kapitel: CLI

12 CLI

12.1 Einführung

Aus `mdstack/tools/mdserver/`, sofern nicht anders angegeben.

12.2 Start und Stop

```
tclsh mdserver.tcl --root /srv/md --port 8080 --bind 127.0.0.1 \
    --trusted-proxy 127.0.0.1 --control 8099 --title Dokumente
tclsh mdctl.tcl --port 8099 ping
tclsh mdctl.tcl --port 8099 status
tclsh mdctl.tcl --port 8099 reload
tclsh mdctl.tcl --port 8099 stop
tclsh mdctl.tcl --host 127.0.0.1 --port 8099 --timeout 5000 stop
```

12.3 PDF-Suche

```
TCLUTILS_TM=/opt/tclutils/lib/tm tclsh mdserver.tcl --root /srv/md
tclsh mdserver.tcl --root /srv/md --pdf-cache /var/cache/mdserver-pdf
tclsh mdserver.tcl --root /srv/md --pdf-search 1     # Warnung, wenn tclutils fehlt
tclsh mdserver.tcl --root /srv/md --pdf-search 0     # nur Markdown
tclsh mdserver.tcl --root /srv/md --office-search 0  # nur Markdown und PDF
tclsh mdserver.tcl --root /srv/md --csv-view 0       # CSV-Dateien unverändert
    ausliefern
tclsh mdserver.tcl --root /srv/md --office-view 0    # Office-Dateien herunterladen
    statt zeigen
tclsh mdserver.tcl --root /srv/md --viewcache 64     # mehr Platz für Office- und
    CSV-Seiten
tclsh mdserver.tcl --root /srv/md --pdf-worker 0     # PDFs im Serverprozess lesen
```

12.4 Grenzen

```
tclsh mdserver.tcl --root /srv/md --maxconn 500 --maxperaddr 64
tclsh mdserver.tcl --root /srv/md --maxperaddr 0      # keine Grenze je Absender
```

12.5 Lokal

```
tclsh mdserver.tcl --root ../../docs --port 8080
tclsh mdserver.tcl --root demo/docs --port 8080
tclsh mdserver.tcl --root ../../docs --bind '*'    # im vertrauten Netz
tclsh mdserver.tcl --help
```

12.6 Stil und Theme

```
tclsh mdserver.tcl --root /srv/md --theme dunkel           # site, dunkel
tclsh mdserver.tcl --root /srv/md --style plain            # früheres Aussehen
tclsh mdserver.tcl --root /srv/md --style sidebar --theme solarized
```

12.7 URLs

```
/?q=control+port                  Volltextsuche (auch PDFs, je Seite)
/?q=                              Suchtipps
/?q=proxy&seite=2                 zweite Seite der Treffer
/?q=proxy&format=json             Treffer als JSON
/tabelle.xlsx?raw=1               die Datei statt der Seite
/?nav=index                       alle Dokumente
/doc.md?theme=dunkel              Dunkelmodus (site)
/doc.md?style=plain&toc=0         älterer Stil ohne TOC
/__mdserver/health                ok oder 503
```

12.8 Zertifikat

```
tclsh mkcert.tcl
tclsh mkcert.tcl --cn mdstack --san 192.168.1.50
tclsh mkcert.tcl --cn mdstack --san 192.168.1.50 --out /etc/mdserver
tclsh mkcert.tcl --check
```

12.9 Tcl/Tk-Handbuch

Aus `mdstack/tools/`:

```
tclsh nroff2md.tcl --batch tcltkdoc/ /srv/md/tcltk/ --linkmode file
```

Nächstes Kapitel: Glossar

13 Glossar

Brotkrumen — Pfadzeile über dem Text im Stil `site`: Start › Ordnertitel › Seitentitel.

Buch-Startseite — von mdserver aus `book.tcl` erzeugte Seite für einen Buchordner ohne `index.md`: Titel, Untertitel, README-Absatz, Downloads, Kapitel.

Clean URL — Pfad ohne `.md`; der Server sucht die Datei mit Endung, auch in Unterordnern.

Control-Port — localhost-Kanal `ping`, `status`, `reload`, `stop` (`mdctl.tcl`).

Dunkelmodus — dunkle Farben im Stil `site`; nach Systemeinstellung oder per `theme=dunkel`.

Horchadresse — Adresse, auf der mdserver Verbindungen annimmt (`--bind`); Vorgabe `127.0.0.1`, `*` für alle Schnittstellen.

Handbuchseite — in Markdown umgewandelte Manpage, erste H1 `# NAME`; mdserver nennt sie nach `CommandName` oder der NAME-Zeile.

Health-Endpunkt — Vorgabe `/__mdserver/health`.

Kachel — Karte für ein Buch, einen Ordner oder ein Dokument auf einer Verzeichnisseite im Stil

`site`.

Nebenläufigkeit — eine Coroutine je Verbindung, non-blocking I/O, alles in einem Thread.

Office-Ansicht — eine `.docx`, `.odt`, `.odp`, `.xlsx` oder `.ods` als Seite statt als Download (`--office-view`).

PDF-Worker — eigener `tclsh` je PDF (`lib/pdftext-worker.tcl`), der den Text für die Suche liest, damit der Server weiter antwortet.

Range — Teil einer Datei (206), für große PDFs.

Suchindex — Text, Titel und Wörter aller durchsuchten Dateien, je Datei mit ihrer mtime; entsteht nach dem Start im Hintergrund.

Suchtipps — aufklappbare Hilfe zur Suchsyntax auf der Ergebnisseite, mit Beispielen aus dem eigenen Bestand.

Stil site — Vorgabe-Seitenstil: ganzes Layout mit Kopfleiste, Seitenleiste, Brotkrumen, "Auf dieser Seite".

trusted-proxy — Gegenstellen, deren `X-Forwarded-For` gilt.

Umlautfaltung — die Suche behandelt `ae`, `oe`, `ue`, `ss` wie `ä`, `ö`, `ü`, `ß`.

Verbindungsgrenze — höchstens so viele offene Verbindungen, insgesamt (`--maxconn`) und je Absender (`--maxperaddr`); darüber 503.

Volltextsuche — `/?q=Wörter` über alle `.md` der Wurzel, mit tclutils auch über PDFs, UND-verknüpft.

Wurzel — `--root`, der Dokumentenbaum.

Zwischenablage — fertige Office- und CSV-Seiten, die liegen bleiben, bis sich die Datei ändert (`--viewcache`).

Nächstes Kapitel: Fehlermeldungen

14 Fehlermeldungen

14.1 Einführung

Meldungen am Start und im Betrieb. `ERROR` und `WARNING` gehen nach stderr, die Meldungen des PDF-Workers ins Protokoll.

14.2 Start

```
ERROR: Cannot bind to HTTP port 8080: couldn't open socket: address already in use
ERROR: Cannot bind to HTTP port 8080 on 127.0.0.1: couldn't open socket: address
    already in use
```

Port belegt; mit `--bind` steht die Adresse dabei. Über den Control-Port der alten Instanz `stop`, sonst den Prozess finden. Nicht blind `fuser -k`, wenn ein anderer Dienst 8080 hat.

```
WARNING: tls not available -- HTTPS disabled: can't find package tls
```

Kein Abbruch. `--cert` und `--key` sind gesetzt, `tcl-tls` fehlt. HTTP läuft weiter, HTTPS nicht.

```
ERROR: Certificate not found: server.crt
ERROR: Key not found: server.key
```

Pfad zu `--cert` oder `--key` falsch. Relativ gilt zum Arbeitsverzeichnis.

```
ERROR: mdserver 0.4 nicht gefunden: can't find package mdserver 0.4
       mdserver-0.4.tm muss in lib/ liegen.
```

Der Launcher findet sein eigenes Modul nicht. `lib/mdserver-0.4.tm` neben `mdserver.tcl` prüfen.

```
ERROR: mdserver::search 0.4 not found (can't find package mdserver::search 0.4).
       lib/mdserver/search-0.4.tm muss neben lib/mdserver-0.4.tm liegen.
```

Die Suche ist ein eigenes Modul. Den Ordner `lib/mdserver/` mit `search-0.4.tm` neben `mdserver-0.4.tm` legen. Dieselbe Meldung mit `mdserver::view` und `view-0.4.tm` heißt: die Office- und CSV-Ansicht fehlt.

```
ERROR: mdparser 0.2 not found: can't find package mdstack::parser 0.2
```

Das mdstack-`lib/` fehlt. Der Launcher sucht es selbst unter `../../lib`; bei einer Installation gehört `mdstack/parser-0.9.2.tm` in das `lib/` neben dem Skript. Die Meldung nennt die verlangte Mindestfassung 0.2; jede neuere 0.x passt.

```
ERROR: mdhtml 0.1 not found: can't find package docir::mdSource
```

`mdstack::html` ist da, docir nicht. Der Launcher sucht ein docir neben mdstack (`../../../docir/lib/tm`). Sonst `docir-0.1.1.tm` und `docir/` in das `lib/` legen. Statt `docir::mdSource` kann auch `docir::html`, `docir::diag` oder `docir::diagram` genannt sein.

```
WARNING: mdstack::theme 0.1 not found -- themes disabled: …
```

Kein Abbruch. `--theme` wirkt dann nicht.

```
WARNING: control port 8099 not bound: …
```

Kein Abbruch, aber `mdctl.tcl` erreicht den Dienst nicht. Port belegt.

```
WARNING: --pdf-search: tclutils::tupdftext not found (TCLUTILS_TM or module path)
    -- PDFs are not searched: can't find package tclutils::tupdftext 0.1
```

Kein Abbruch. Nur mit `--pdf-search 1`; bei `auto` steht es allein im Banner (`PDF: Suche aus: …`). tclutils in `lib/` legen oder `TCLUTILS_TM` setzen (Kapitel *Starten*).

```
WARNING: --office-search: not found: tclutils::tudocx, tclutils::tuxlsx
WARNING: --csv-view: tclutils::tucsv 0.2 not found -- CSV files are sent as they
    are: …
```

Kein Abbruch. Nur mit `--office-search 1` bzw. `--csv-view 1`; bei `auto` stehen die Formate allein im Banner (`Office:`, `CSV:`). Die erste Zeile heißt: ODF wird gesucht, Word und Excel nicht, weil das tclutils älter als 0.69.0 ist.

14.3 PDF-Worker

Diese Zeilen stehen im Protokoll (stdout, mit Uhrzeit davor), nicht auf stderr. Mit `--no-log` fehlen sie.

```
pdf worker: starpack -- reading in the server
pdf worker: lib/pdftext-worker.tcl missing -- reading in the server
pdf worker: tclutils::tutask not found -- reading in the server
```

Kein Worker; mdserver liest die PDFs im eigenen Prozess wie mit `--pdf-worker 0`. Steht beim Start, wenn die PDF-Suche an ist.

```
pdf worker: /srv/md/a.pdf has no text, reader failed: …
```

`tupdftext` konnte die Datei nicht lesen (beschädigt, keine PDF). Sie steht leer im Cache, bis sie sich ändert.

```
pdf worker failed on /srv/md/a.pdf (…) -- the next search tries again
pdf worker failed again on /srv/md/a.pdf (…) -- skipped until it changes
```

Der Worker ist abgestürzt, endete mit einem Fehlercode oder brauchte länger als 10 Minuten. Beim zweiten Mal bleibt die PDF draußen, bis sie sich ändert.

14.4 mdctl

```
mdctl: 127.0.0.1:8099 nicht erreichbar: couldn't open socket: connection refused
```

Kein mdserver mit `--control 8099` aktiv, oder er läuft auf einem anderen Port.

```
mdctl: keine Antwort von 127.0.0.1:8099
```

Verbindung kam zustande, aber innerhalb von `--timeout` (Vorgabe 10 s) keine Antwort. Rückgabe 1.

14.5 HTTP-Status

Status	Bedeutung
301	Verzeichnis ohne Schrägstrich
304	unverändert
400	im Pfad ein % ohne zwei Hexziffern, %00 oder NUL
403	Pfad außerhalb der Wurzel (auch Symlink hinaus), Punkt-Datei bei --dotfiles 0
404	nicht gefunden
405	Methode nicht GET/HEAD
414	Anfragezeile länger als --maxline
416	Bereich nicht erfüllbar (Anfang hinter Dateiende); unlesbarer Range-Kopf gibt 200
431	eine Kopfzeile länger als --maxline oder alle Köpfe länger als --maxheader
501	?format=json ohne tclutils::tujson: format=json needs tclutils::tujson

Status	Bedeutung
503	Health: Wurzel nicht lesbar; zu viele offene Verbindungen (--maxconn, --maxperaddr), mit Retry-After: 1

14.6 Zertifikat

```
GUELTIG, ABER OHNE subjectAltName
```

Browser lehnen ab. Neu erzeugen, SAN setzen. `--check` endet mit Rückgabe 1.

15 Lizenz


```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen
Bedingungen 4.0 International (CC BY-SA 4.0)
Codebeispiele des Autors: MIT-Lizenz (unten)

---

Text

Dieses Buch ist unter der Creative-Commons-Lizenz
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0) lizenziert.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen
und weiterverbreiten sowie es remixen, verändern und darauf aufbauen,
auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben
  machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen
  vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material
  remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen
  Sie Ihre Beiträge nur unter derselben Lizenz wie das Original
  verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

---

Codebeispiele

Die Codebeispiele des Autors (in den Codeblöcken des Buchs und als
Skripte daneben) stehen zusätzlich unter der MIT-Lizenz. Übernommene
Codebeispiele und zitierte Texte aus anderen Projekten (Tcl/Tk,
pdf4tcl und andere) behalten die Lizenz ihres Originals; die Quelle ist
jeweils angegeben.
```

```
MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

B

Brotkrumen 15
Buch-Startseite 14

C

Clean URL 22

D

docir-Module 6
Dunkelmodus 16

E

Eingrenzen 18

H

Handbuchseite 15
Hilfe auf der Ergebnisseite 18

J

JSON 21

K

Kachel 16

L

Launcher 8

M

mdserver 5

O

Office-Datei 17

S

Stil site 12
Suchindex 19
Suchsyntax 18

T

Tabelle 17
tclutils::tupdftext 19

U

Umlautfaltung 18

V

Verbindungsgrenzen 23
Volltextsuche 14
Vorgabe 7

W

Wurzel 7

Z

Zwischenablage 26